1. INTRODUCTION

Artificial Intelligence (AI) has become one of the most significant fields in computer science and has found applications in various domains such as healthcare, robotics, autonomous vehicles, expert systems, natural language processing, and game development. AI enables computer systems to perform tasks that normally require human intelligence, including reasoning, decision making, problem solving, learning, and planning. Among the various application areas of AI, game development provides an excellent platform for implementing and analyzing intelligent agent behavior in dynamic environments.

Games have long been used as test environments for Artificial Intelligence research because they involve decision making, strategic planning, problem solving, and real-time interaction. Various AI techniques such as search algorithms, pathfinding algorithms, state-space representation, inference mechanisms, and multi-agent systems can be effectively demonstrated through gaming applications. Game environments allow researchers and developers to study intelligent behavior under controlled conditions.

Pacman is one of the most popular arcade games and has been extensively used in Artificial Intelligence education and research. The traditional Pacman game consists of a player-controlled character that moves through a maze while avoiding enemy ghosts and collecting food pellets. In conventional implementations, ghost movements often follow predefined patterns, making the game behavior predictable. However, introducing AI techniques into the game environment can significantly improve the intelligence and behavior of these agents.

The Smart Pacman AI project focuses on the development of an intelligent gaming environment using classical Artificial Intelligence techniques. In this system, Pacman is controlled manually by the player, while the ghost agents act autonomously using different AI strategies. The objective is to create a realistic and intelligent game environment where multiple agents interact dynamically and make decisions based on environmental conditions.

The proposed system uses a Multi-Agent AI approach where each ghost behaves differently according to its assigned intelligence. The Red Ghost uses the A-star search algorithm to determine the shortest path toward Pacman and continuously attempts to chase the player. The Pink Ghost uses predictive behavior to estimate the future position of Pacman and attempts to intercept the player strategically. The Blue Ghost introduces randomness into the environment by moving unpredictably, thereby increasing the complexity and uncertainty of the game.

The A-star search algorithm is one of the most efficient pathfinding algorithms used in Artificial Intelligence and game development. It combines the advantages of uniform-cost search and heuristic search to determine the optimal path between two locations. In the Smart Pacman AI system, A-star enables the Red Ghost to navigate the maze intelligently while avoiding obstacles and minimizing travel distance.

The project also utilizes Finite State Machines (FSM) to control ghost behaviors. Different states such as CHASE, SCATTER, and FRIGHTENED are implemented to simulate dynamic behavior changes. When Pacman consumes a power pellet, the ghosts temporarily enter the

frightened state and move away from Pacman. These state transitions make the gameplay more realistic and demonstrate adaptive AI behavior.

Knowledge representation and inference mechanisms play an important role in the proposed system. The maze structure, wall locations, ghost positions, Pacman position, and game states collectively form the knowledge base of the system. The ghost agents continuously analyze this information and infer appropriate actions such as chasing, predicting, escaping, or random movement. These inference mechanisms allow the agents to behave autonomously without direct human intervention.

The Smart Pacman AI project also demonstrates the concept of Rule-Based Artificial Intelligence. Several IF-THEN rules are implemented to control movement, collision detection, score calculation, frightened mode activation, and behavioral transitions. These rules enable the agents to make decisions according to the current state of the game environment.

The project has been developed using Python programming language and the Pygame library. Python provides simplicity and flexibility for implementing AI algorithms, while Pygame offers efficient tools for graphics rendering, animation, event handling, and game development. The modular architecture of the system allows different AI components to operate independently while interacting within the game environment.

The major objective of this project is to demonstrate how classical Artificial Intelligence techniques can be integrated into a real-time gaming environment to create intelligent agents capable of autonomous decision making. The project provides practical implementation of various AI concepts such as:

- Search algorithms
- State-space representation
- Knowledge representation
- Inference mechanisms
- Multi-agent systems
- Rule-based reasoning
- Finite State Machines

The Smart Pacman AI system successfully demonstrates how intelligent agents can interact within a dynamic environment and make decisions based on changing conditions. The project serves as an educational platform for understanding the practical applications of Artificial Intelligence concepts in game development and autonomous systems.

Thus, the Smart Pacman AI project provides an effective implementation of classical AI techniques and demonstrates how intelligent behavior can be achieved through search algorithms, rule-based systems, and multi-agent architectures in a real-time gaming environment.

2. SYSTEM ARCHITECTURE

The Smart Pacman AI system is designed using a modular architecture in which multiple components interact to create an intelligent game environment. The system integrates game mechanics with Artificial Intelligence techniques such as A-star search, rule-based reasoning, finite state machines, and multi-agent systems.

The overall architecture consists of five major layers:

1. User Input Layer
2. Environment Layer
3. Artificial Intelligence Layer
4. Game Processing Layer
5. Rendering Layer

These layers work together to provide real-time interaction between the player and intelligent ghost agents.

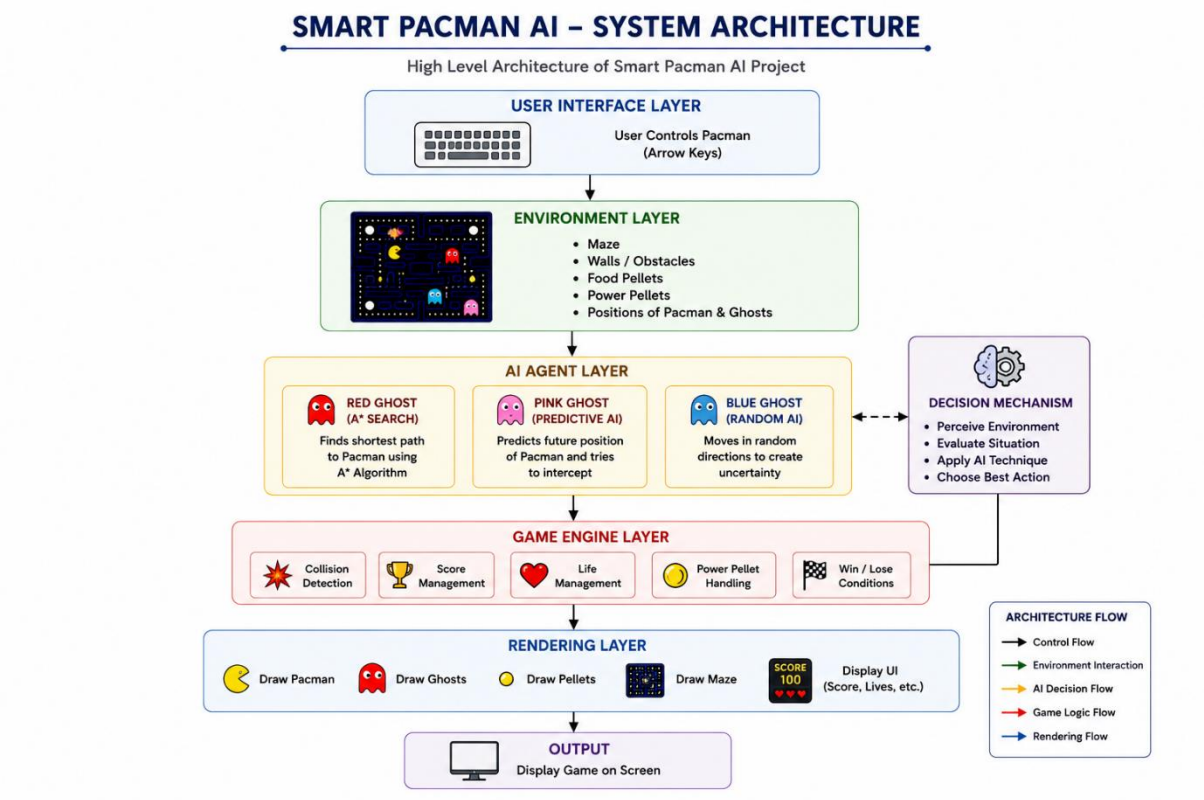

Fig1:System Architecture

The system architecture of Smart Pacman AI consists of multiple layers that work together to create an intelligent gaming environment. The architecture follows a layered approach where user interaction, environment representation, artificial intelligence agents, game logic, and rendering modules communicate to produce real-time gameplay.

- **User Interface Module**

The User Interface Layer allows the player to interact with the game using keyboard controls. The arrow keys are used to control the movement of Pacman within the maze. This layer acts as the communication interface between the user and the game system.

- **Environment Module**

The Environment Layer represents the game world. It contains the maze structure, walls, obstacles, food pellets, power pellets, and the positions of Pacman and the ghost agents. This layer provides environmental information required by the intelligent agents to make decisions.

- **AI Agent Module**

The AI Agent Layer is the core intelligence component of the system. It consists of three autonomous ghost agents:

- Red Ghost: Uses the A-star search algorithm to determine the shortest path toward Pacman.
- Pink Ghost: Uses predictive behavior to estimate the future position of Pacman and attempts interception.
- Blue Ghost: Uses random movement to create uncertainty and dynamic gameplay.

Each ghost operates independently and makes decisions according to its assigned behavior.

- **Decision Mechanism**

The decision mechanism enables the ghost agents to observe the environment, analyze the current situation, apply appropriate AI techniques, and select suitable actions. The agents continuously evaluate the game state and update their movements accordingly.

- **Game Engine Module**

The Game Engine Layer controls the internal game operations. It performs collision detection, score management, life management, power pellet handling, and win or loss condition evaluation. This layer ensures that the game rules are properly executed.

- **Rendering Module**

The Rendering Layer is responsible for displaying the game elements on the screen. It draws the maze, Pacman, ghost agents, pellets, scores, and user interface components. The rendering process continuously updates the display to provide real-time gameplay.

- **Output Module**

The Output Layer presents the final game screen to the player. It displays the current game state, score, remaining lives, and the movements of the intelligent agents.

Overall, the proposed architecture integrates user interaction, environment modeling, artificial intelligence techniques, and game processing mechanisms to create an intelligent and interactive Pacman gaming system.

2.2 DATAFLOW

The Dataflow of the Smart Pacman AI system illustrates the sequence of operations performed during the execution of the game. The process begins with user input and continues through AI decision making, game state updates, and rendering until the game reaches a win or lose condition.

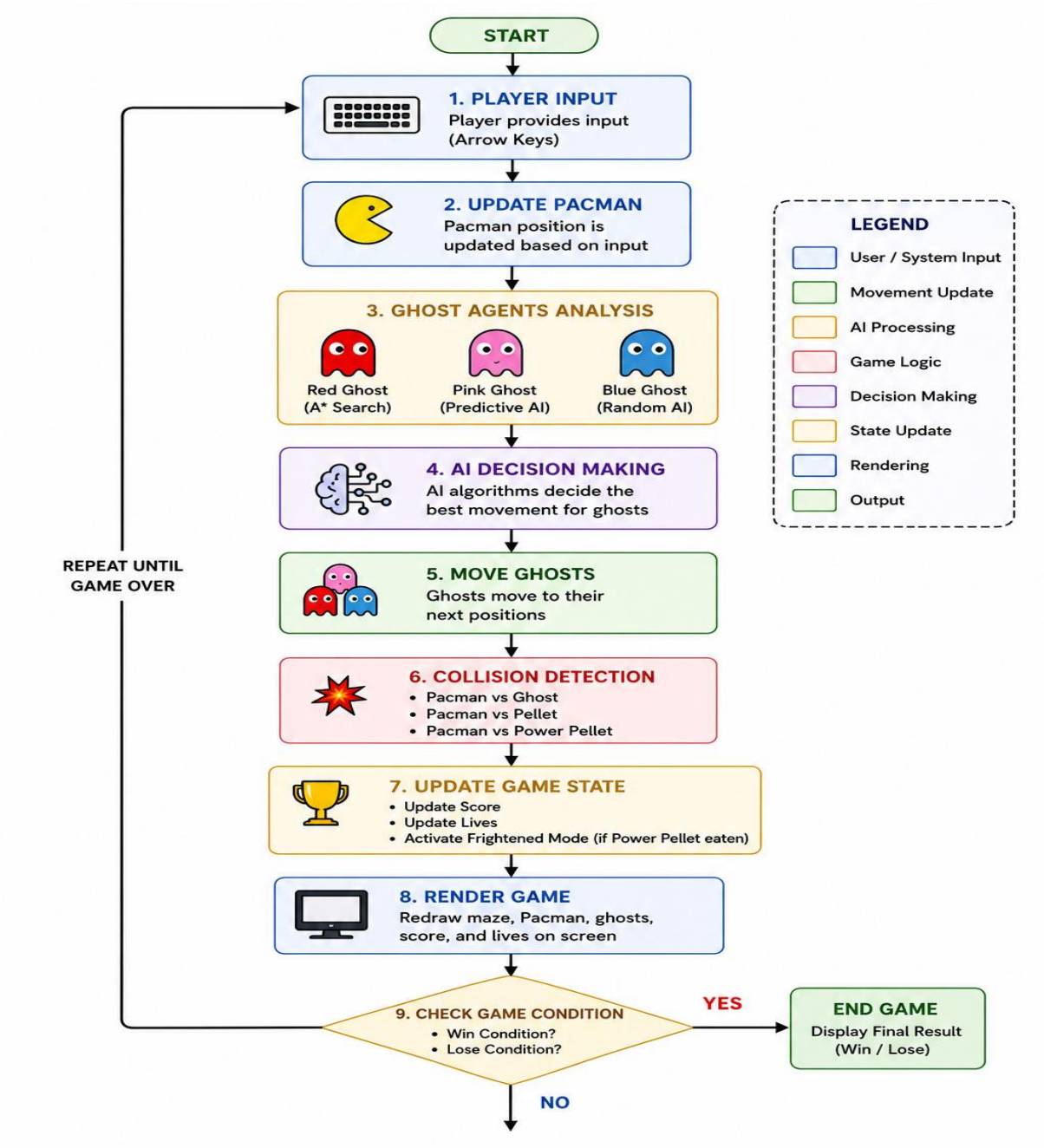

Fig2:Dataflow

Step 1: Player Input

The Dataflow begins when the player provides input using the keyboard arrow keys. The user controls the movement of Pacman by selecting one of the four possible directions: up, down, left, or right.

Step 2: Update Pacman Position

Based on the user's input, the position of Pacman is updated within the maze environment. Before movement, the system checks whether the next cell contains a wall. If the cell is valid, Pacman moves to the new location.

Step 3: Ghost Agents Analysis

After updating Pacman's position, all ghost agents analyze the current game environment. Each ghost uses a different AI behavior:

- Red Ghost uses the A-star search algorithm.
- Pink Ghost uses predictive AI.
- Blue Ghost uses randomized movement.

The ghost agents observe the position of Pacman, maze structure, and game conditions before making decisions.

Step 4: AI Decision Making

The AI algorithms process the available information and determine the most suitable movement for each ghost. The Red Ghost calculates the shortest path, the Pink Ghost predicts future movement, and the Blue Ghost selects a random valid direction.

Step 5: Move Ghosts

After the decision-making process, the ghost agents move to their next positions according to their respective algorithms and behavioral rules.

Step 6: Collision Detection

The system continuously checks for collisions between:

- Pacman and ghosts.
- Pacman and food pellets.
- Pacman and power pellets.

If a collision with a ghost occurs, the player loses a life. If Pacman collects pellets, the score increases.

Step 7: Update Game State

The game state is updated after every movement. This includes:

- Updating the score.
- Updating the remaining lives.
- Activating frightened mode when a power pellet is consumed.
- Updating ghost states.

Step 8: Render Game

The rendering module redraws the maze, Pacman, ghost agents, pellets, score, and remaining lives on the screen. The display is continuously updated to provide smooth real-time gameplay.

Step 9: Check Game Condition

The system checks whether:

- All lives are lost.
- All pellets have been collected.

If either condition is satisfied, the game terminates and displays the final result. Otherwise, the workflow repeats until the game ends.Thus, the Smart Pacman AI Dataflow demonstrates the interaction between user input, intelligent agents, AI algorithms, and game logic to create an autonomous and dynamic gaming environment.

2.3 MULTI-AGENT ARCHITECTURE

The project implements a Multi-Agent AI system.

The agents are:

Pacman Agent

- Controlled by player.

Red Ghost Agent

- Uses A-star search.
- Aggressive chasing behavior.

Pink Ghost Agent

- Predicts future movement.
- Attempts interception.

Blue Ghost Agent

- Uses randomized movement.
- Creates uncertainty.

Each agent acts independently and makes decisions according to its own behavior.

3. METHODOLOGY

The Smart Pacman AI system follows a rule-based and search-based Artificial Intelligence methodology. The objective of the proposed methodology is to create intelligent ghost agents capable of autonomous decision making within a dynamic gaming environment.

The methodology consists of several stages including environment creation, state representation, knowledge representation, inference mechanisms, pathfinding, behavior control, and game state updates.

The overall methodology enables the ghost agents to perceive the environment, analyze the available information, infer suitable actions, and perform intelligent movements.

3.1 ENVIRONMENT MODELING

The game environment is represented as a two-dimensional grid-based maze. The maze consists of:

- Walls (#)
- Empty spaces
- Food pellets (.)
- Power pellets (O)
- Pacman position
- Ghost positions

Each cell of the grid represents a particular state of the environment.

The maze acts as the knowledge base for all intelligent agents.

The agents use this information to:

- identify valid paths,
- avoid obstacles,
- locate Pacman,
- determine movement decisions.

3.2 ALOGARITHM

A state represents the complete condition of the game at a particular instant.

The state is represented as:

S = {P(x,y), R(x,y), Pi(x,y), B(x,y), Mode, Score, Lives}

Where:

- P(x,y) → Pacman position
- R(x,y) → Red Ghost position
- Pi(x,y) → Pink Ghost position
- B(x,y) → Blue Ghost position
- Mode → Ghost behavior state
- Score → Current score
- Lives → Remaining lives

Each movement of Pacman or a ghost produces a new state.

The collection of all possible states forms the state space of the game.

3.3 KNOWLEDGE REPRESENTATION

Knowledge representation refers to storing information required by the intelligent agents.

The Smart Pacman AI system stores knowledge in the following forms:

Maze Representation

The maze structure stores:

- walls
- paths
- pellets
- power pellets

Position Representation

The positions of:

- Pacman
- Red Ghost
- Blue Ghost
- Pink Ghost

are continuously maintained.

Behavioral Knowledge

Ghost states:

- CHASE
- SCATTER

- FRIGHTENED

represent behavioral knowledge.

Movement Rules

IF-THEN rules are used to represent movement decisions.

3.4 INFERENCE MECHANISM

Inference is the process of making decisions using available knowledge.

The ghost agents continuously analyze:

- Pacman position
- Maze structure
- Ghost state
- Movement direction

Based on this information, they infer the best action.

Examples:

- Red Ghost infers shortest path.
- Pink Ghost infers future position.
- Blue Ghost infers random movement.
- Frightened ghosts infer escape behavior.

Inference allows the agents to behave autonomously.

3.5 RULE-BASED SYSTEM

The Smart Pacman AI project uses several IF-THEN rules.

Examples:

Rule 1:
IF the next cell is a wall
THEN movement is blocked.

Rule 2:
IF Pacman collects a pellet
THEN score increases.

Rule 3:
IF Pacman eats a power pellet
THEN frightened mode is activated.

Rule 4:
IF ghost mode is CHASE
THEN use A-star pathfinding.

Rule 5:
IF frightened mode is ON
THEN ghosts move away from Pacman.

Rule 6:
IF collision occurs
THEN lives decrease.

These rules provide decision-making capabilities.

3.6 A-STAR SEARCH ALGORITHM

The Red Ghost uses the A-star search algorithm.

A-star is a heuristic search algorithm that finds the shortest path between two positions.

The evaluation function is:

$f(n) = g(n) + h(n)$

Where:

- $g(n)$ = actual path cost
- $h(n)$ = estimated cost to target
- $f(n)$ = total cost

The algorithm performs the following steps:

1. Take ghost position as start node.
2. Take Pacman position as goal node.
3. Generate neighboring nodes.
4. Calculate costs.
5. Select minimum-cost node.
6. Repeat until goal is reached.
7. Return shortest path.

The Red Ghost follows this path to chase Pacman.

3.7 PREDICTIVE AI

The Pink Ghost uses predictive behavior.

Instead of moving toward the current position of Pacman, it estimates the future position.

The prediction depends on:

- movement direction,
- current position,
- prediction distance.

This allows the ghost to intercept Pacman.

3.8 RANDOMIZED AI

The Blue Ghost uses randomized movement.

The movement directions:

- Up
- Down
- Left
- Right

are randomly selected.

Random movement creates:

- uncertainty,
- dynamic gameplay,
- unpredictable behavior.

3.9 FINITE STATE MACHINE

Ghost behaviors are controlled using a Finite State Machine.

States:

CHASE

Ghost aggressively pursues Pacman.

SCATTER

Ghost movement becomes less aggressive.

FRIGHTENED

Ghost moves away from Pacman.

State transitions occur according to:

- timer values,
- power pellet activation,

- game conditions.

3.10 MULTI-AGENT SYSTEM

The Smart Pacman AI system contains multiple autonomous agents.

Agents:

1. Pacman Agent
2. Red Ghost Agent
3. Pink Ghost Agent
4. Blue Ghost Agent

Each agent has:

- independent behavior,
- separate decision making,
- individual objectives.

The agents interact within a common environment.

This forms a Multi-Agent AI system.

4.EXPERIMENTAL SETUP

The experimental setup describes the hardware, software, development tools, and testing environment used for implementing the Smart Pacman AI system.

4.1 Hardware Requirements

The Smart Pacman AI system was developed and tested using the following hardware configuration:

Component	Specification
Processor	Intel Core i3 or higher
RAM	4 GB or above
Storage	500 MB free space
Display	1366 × 768 resolution
Keyboard	Standard keyboard

4.2 Software Requirements

The following software tools were used for the implementation:

Software	Purpose
Windows 10/11	Operating System
Python 3.13	Programming Language
Pygame	Game development library
Visual Studio Code	Code editor

4.3 Development Tools

The Smart Pacman AI project was developed using Python and Pygame. Python was chosen because of its simplicity and extensive support for Artificial Intelligence algorithms. Pygame provides functions for graphics rendering, event handling, animations, and game development

4.4 AI Techniques Used

The following Artificial Intelligence techniques were implemented in the project:

- A-star Search Algorithm
- Finite State Machine
- Rule-Based System
- Predictive AI
- Multi-Agent System
- Knowledge Representation
- Inference Mechanism

4.5 Testing Environment

The system was tested under various game conditions including:

- Normal gameplay
- Ghost chasing behavior
- Power pellet activation
- Frightened mode
- Collision detection

- Score calculation
- Game over condition
- Win condition

Multiple test runs were performed to verify the behavior of the ghost agents and ensure proper functioning of the AI algorithms.

4.6 Experimental Observations

The Red Ghost successfully used the A-star algorithm to determine the shortest path toward Pacman.

The Pink Ghost predicted the future movement direction of Pacman and attempted interception.

The Blue Ghost introduced random movement behavior, increasing the uncertainty of the game.

The frightened mode changed the ghost behavior dynamically after Pacman consumed a power pellet.

The experimental results demonstrated that the intelligent agents were able to make autonomous decisions and respond to changing game conditions in real time.

5. IMPLEMENTATION

The Smart Pacman AI system was implemented using Python and the Pygame library. The implementation integrates game mechanics with Artificial Intelligence techniques to create intelligent ghost agents capable of autonomous decision making.

The system consists of multiple modules that interact to control player movement, ghost behavior, score management, collision detection, and game state transitions.

5.1 GAME ENVIRONMENT IMPLEMENTATION

The game environment is represented as a two-dimensional grid-based maze. The maze contains:

- Walls (#)
- Empty paths
- Food pellets (.)
- Power pellets (O)
- Pacman
- Ghost agents

The maze acts as the environment in which all agents operate.

Movement is restricted by walls, and agents can only move through valid paths.

5.2 AGENT IMPLEMENTATION

The project contains four agents:

Pacman Agent

- Controlled by the player.
- Collects pellets.
- Avoids ghosts.

Red Ghost Agent

- Uses A-star search.
- Finds the shortest path.
- Aggressively chases Pacman.

Pink Ghost Agent

- Predicts future movement.
- Attempts to intercept Pacman.

Blue Ghost Agent

- Uses randomized movement.
- Creates uncertainty in gameplay.

Each agent operates independently.

5.3 RULE IMPLEMENTATION

The Smart Pacman AI system uses several IF-THEN rules.

Rule 1:

IF the next cell contains a wall
THEN movement is blocked.

Rule 2:

IF Pacman touches a food pellet
THEN score increases.

Rule 3:

IF Pacman consumes a power pellet
THEN frightened mode is activated.

Rule 4:

IF the Red Ghost enters chase mode
THEN A-star pathfinding is executed.

Rule 5:

IF frightened mode is active
THEN ghosts move away from Pacman.

Rule 6:

IF a ghost collides with Pacman
THEN one life is reduced.

Rule 7:

IF lives become zero
THEN the game ends.

Rule 8:

IF all pellets are collected
THEN the player wins.

5.4 ACTION IMPLEMENTATION

The agents perform different actions during gameplay.

Pacman Actions

- Move Up
- Move Down
- Move Left
- Move Right
- Collect pellets
- Consume power pellets

Red Ghost Actions

- Analyze Pacman position.
- Calculate shortest path.
- Chase Pacman.

Pink Ghost Actions

- Predict future position.
- Intercept Pacman.

Blue Ghost Actions

- Select random movement.
- Explore the maze.

Frightened Actions

- Move away from Pacman.
- Avoid collision.

5.5 A-STAR IMPLEMENTATION

The Red Ghost uses the A-star search algorithm.

The algorithm:

1. Takes ghost position as the start node.
2. Takes Pacman position as the goal node.
3. Generates neighboring cells.
4. Calculates path costs.
5. Selects the optimal path.
6. Returns the shortest route.

The Red Ghost then moves one step along this path.

This enables intelligent chasing behavior.

5.6 FINITE STATE MACHINE IMPLEMENTATION

The ghost behavior is controlled using three states:

CHASE

The ghost pursues Pacman.

SCATTER

The ghost moves less aggressively.

FRIGHTENED

The ghost moves away from Pacman after a power pellet is consumed.

State transitions occur according to:

- timers,
- game conditions,
- power pellet activation.

5.7 COLLISION DETECTION

The system continuously checks:

- Pacman and ghost collision.
- Pellet collection.
- Power pellet collection.

When collisions occur:

- score is updated,
- lives decrease,
- game states change.

5.8 STATE SPACE TREE

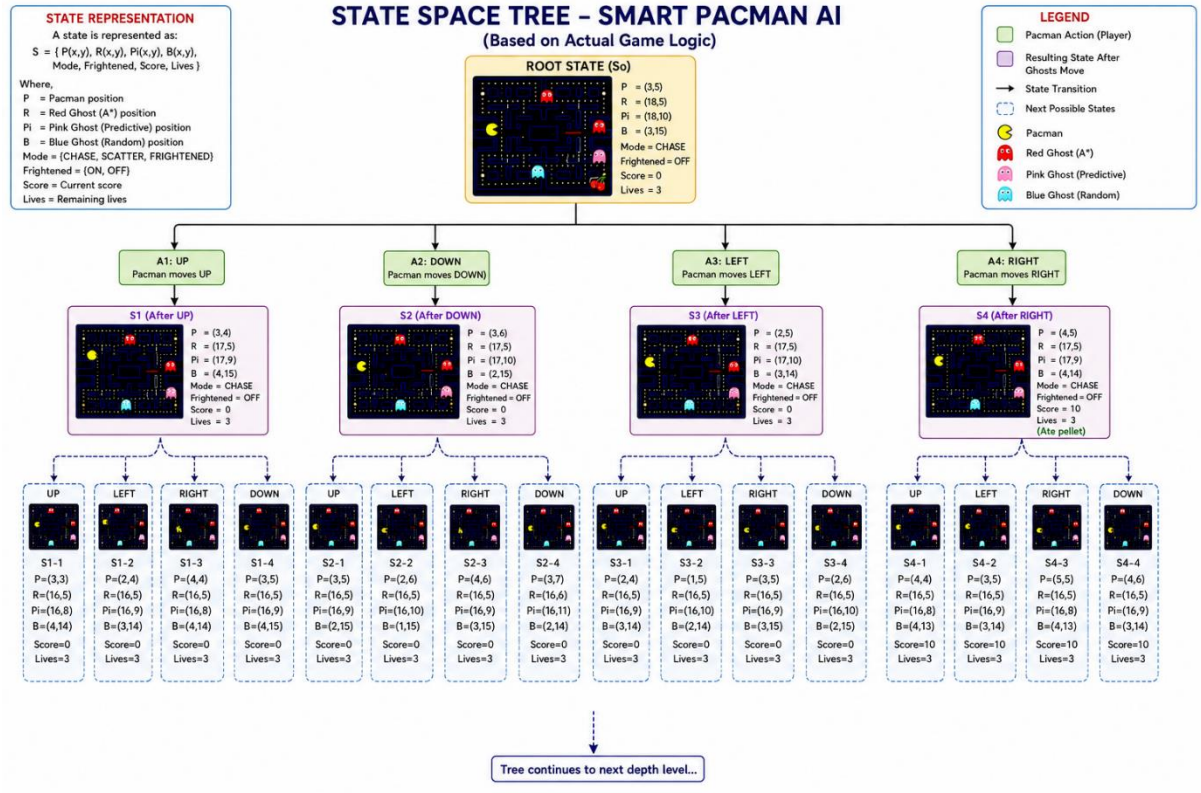

Fig3:State Space Tree

The state space tree represents the various possible states generated during the execution of the Smart Pacman AI system. A state is defined as the complete configuration of the game at a particular instant, including the positions of Pacman and all ghost agents, ghost behavior mode, frightened state, score, and remaining lives.

The state representation used in the system is:

S = {P(x,y), R(x,y), Pi(x,y), B(x,y),

Mode, Frightened, Score, Lives}

where:

- P = Pacman position
- R = Red Ghost position
- Pi = Pink Ghost position
- B = Blue Ghost position
- Mode = Current ghost mode
- Frightened = Frightened state
- Score = Current score
- Lives = Remaining lives

Root State (S_0)

The root state represents the initial configuration of the game before any movement occurs. At this state:

- Pacman occupies its starting position.
- The Red Ghost is initialized in chase mode.
- The Pink Ghost occupies its predictive position.
- The Blue Ghost occupies its random movement position.
- The score is zero.
- The frightened mode is disabled.
- Three lives are available.

This state serves as the starting point for all future state transitions.

Pacman Actions

From the root state, Pacman can perform four possible actions:

- Move Up

- Move Down
- Move Left
- Move Right

These actions generate the states:

- S1 (After UP)
- S2 (After DOWN)
- S3 (After LEFT)
- S4 (After RIGHT)

Each movement creates a new game state.

Ghost State Transitions

After Pacman performs an action, the ghost agents analyze the updated game environment and determine their next positions.

Red Ghost (A*)

The Red Ghost uses the A-star search algorithm to calculate the shortest path toward Pacman. It evaluates neighboring cells and selects the movement with the minimum path cost.

Pink Ghost (Predictive AI)

The Pink Ghost predicts the future position of Pacman based on the current movement direction and attempts to intercept the player.

Blue Ghost (Random AI)

The Blue Ghost selects a random valid direction, introducing uncertainty and unpredictable behavior.Thus, every Pacman action produces a new state, and every ghost decision further expands the state space.

Level 1 States

The states S1, S2, S3, and S4 represent the first level of the tree. These states are generated immediately after Pacman performs one movement action.The ghost agents update their positions according to their respective AI algorithms, resulting in different game configurations.

Level 2 States

Each Level 1 state generates additional child states such as:

- S1-1
- S1-2

- S1-3
- S1-4

These states represent subsequent Pacman movements and ghost responses.

At every level:

- Pacman changes position.
- Ghosts update positions.
- Scores may increase.
- Modes may change.
- Game conditions are evaluated.

State Expansion

The state space continues to expand as long as the game is active. Every movement creates a new node in the state tree.The expansion continues until one of the following terminal conditions is reached:

Win State

The game reaches a goal state when all pellets are collected.

Lose State

The game reaches a failure state when Pacman loses all lives after colliding with ghost agents.

Importance of State Space Representation

The state space tree provides several advantages:

- Represents all possible game states.
- Illustrates AI decision-making.
- Shows agent interactions.
- Demonstrates state transitions.
- Helps analyze ghost behavior.
- Visualizes search and exploration.

Thus, the state space tree provides a clear representation of how the Smart Pacman AI system generates and explores different game situations using multiple intelligent agents and Artificial Intelligence techniques.

Example State Transition

Consider the root state:

So

P = (3,5)

R = (18,5)

Pi = (18,10)

B = (3,15)

Pacman performs the action:

A4 : RIGHT

New state:

S4

P = (4,5)

R = (17,5)

Pi = (17,9)

B = (4,14)

The Red Ghost moves one step toward Pacman using A*, the Pink Ghost predicts the next position, and the Blue Ghost moves randomly. This generates a new game state.This process continues until the game reaches either the win state or the lose state.

6.RESULTS

The Smart Pacman AI system was successfully implemented and tested using Python and the Pygame library. The game environment operated efficiently and demonstrated intelligent behavior through multiple autonomous ghost agents.

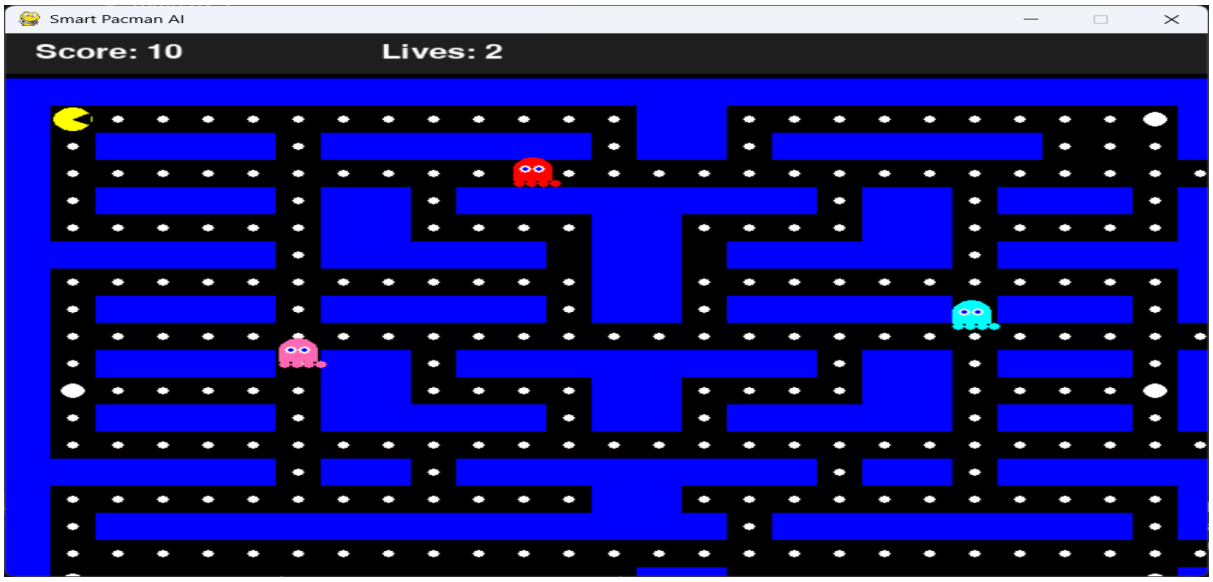

Fig4:SMART PACMAN AI

The Red Ghost successfully utilized the A-star search algorithm to calculate the shortest path toward Pacman. The ghost was able to navigate around walls and obstacles and continuously pursue the player using intelligent pathfinding techniques.

The Pink Ghost implemented predictive behavior by estimating the future movement direction of Pacman and attempting to intercept the player. This behavior increased the strategic complexity of the game.

The Blue Ghost introduced randomness into the environment by selecting movement directions unpredictably. This created uncertainty and prevented the gameplay from becoming repetitive.

The Finite State Machine successfully controlled ghost behavior through CHASE, SCATTER, and FRIGHTENED states. When Pacman consumed a power pellet, the ghosts entered frightened mode and temporarily moved away from the player.

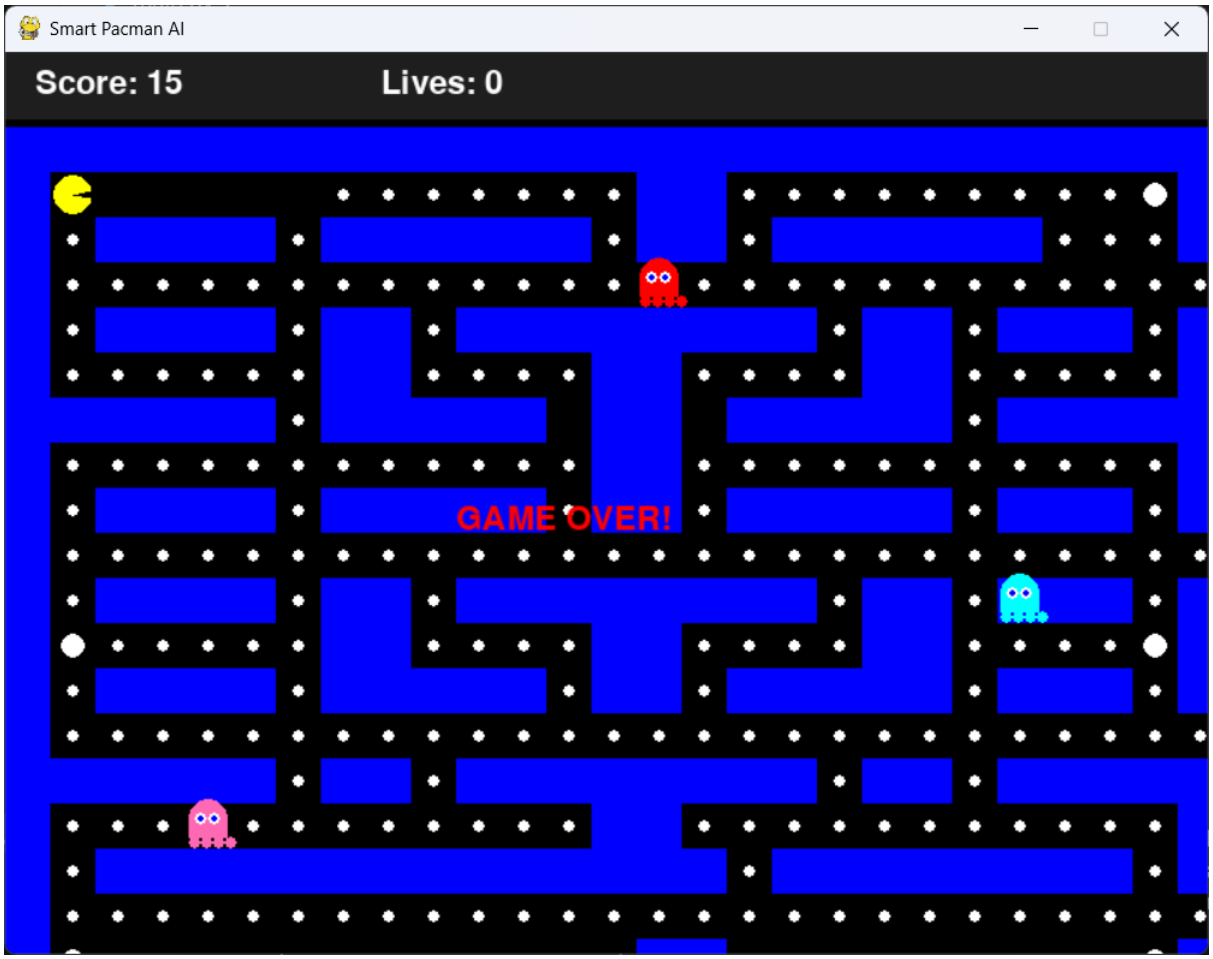

Fig5:SMART PACMAN AI

The lives system, score calculation, collision detection, and game state transitions operated correctly during gameplay. The game successfully demonstrated real-time decision making, autonomous agent behavior, and intelligent pathfinding.

The experimental results show that the Smart Pacman AI system effectively integrates classical Artificial Intelligence techniques to create an interactive and intelligent gaming environment.

7.CONCLUSION

The Smart Pacman AI project successfully demonstrates the application of classical Artificial Intelligence techniques in a real-time gaming environment. The project integrates multiple AI concepts such as A-star search, Finite State Machines, rule-based reasoning, inference mechanisms, and multi-agent systems to create intelligent ghost agents.

The Red Ghost uses the A-star algorithm to perform intelligent pathfinding, while the Pink Ghost applies predictive behavior to estimate the future movement of Pacman. The Blue Ghost introduces randomized movement to increase gameplay complexity and uncertainty.

The implementation of multiple autonomous agents enables the system to demonstrate cooperative behavior, real-time decision making, and dynamic interactions within the game environment. The frightened mode and state transitions further improve the adaptability of the ghost agents.

The project successfully illustrates how search algorithms, knowledge representation, inference, and rule-based systems can be applied to develop intelligent game agents. The Smart Pacman AI system serves as an effective example of practical Artificial Intelligence implementation in game development.

Overall, the project achieves its objectives by creating an intelligent and interactive gaming environment that demonstrates the fundamental concepts of Artificial Intelligence and autonomous agent behavior.